

Окей, давай теперь отнестись к ноуту как к “универсальному станку” и разберёмся:

1. Где и как его запускать
 2. Какой нужен формат данных (корпус, queries, сабмит)
 3. Что и где менять, когда адаптируешь под новый датасет/соревнование
 4. Как аккуратно править сабмит
-

1. Где запускать ноутбук

Вариант А: Kaggle

Это самый естественный вариант, потому что ноут изначально под Kaggle-хакатон заточен.

Что делать:

1. Заливаешь данные (корпус + queries.json) в Kaggle Dataset или берёшь уже готовый.
2. В ячейке CONFIG меняешь:

```
DATA_PATHS = [  
    "/kaggle/input/ТВОЙ_DATASET/папка_с_доками",  
]
```

```
QUERIES_PATH = "/kaggle/input/ТВОЙ_DATASET/queries.json"
```

- 2.
 3. В настройках ядра:
 - Accelerator: GPU (для Qwen / unsloth),
 - Python 3 + последние библиотеки.
 4. Выполняешь ноут сверху вниз – в конце получишь submission.csv рядом с ноутбуком.
-

Вариант Б: Google Colab

1. В Colab:
 - монтируешь Google Drive или загружаешь папку с документами / queries.json.
2. В CONFIG ставишь абсолютные пути, например:

```
DATA_PATHS = ["/content/drive/MyDrive/rag_docs"]
QUERIES_PATH = "/content/drive/MyDrive/rag_docs/queries.json"
SUBMISSION_CSV = "/content/drive/MyDrive/rag_docs/submission.csv"
```

2.

3. В первой ячейке (где импорты) можно добавить:

```
!pip install rank-bm25 sentence-transformers faiss-cpu langchain-community
langchain-text-splitters langchain-experimental
!pip install transformers accelerate bitsandbytes
```

3.

4. В меню Colab включаешь Runtime → Change runtime type → GPU.

Вариант В: Локально (Jupyter / VSCode)

1. Создаёшь виртуальное окружение, ставишь зависимости:

```
pip install jupyter
pip install rank-bm25 sentence-transformers faiss-cpu langchain-community
langchain-text-splitters langchain-experimental transformers bitsandbytes
```

1.

2. Кладёшь корпус и queries.json в удобную папку, пути в CONFIG ставишь что-то типа:

```
DATA_PATHS = ["/data/corpus"]
QUERIES_PATH = "/data/queries.json"
SUBMISSION_CSV = "/data/submission.csv"
```

2.

3. Запускаешь jupyter notebook, открываешь DIMA_rag-everything_CONFIG_fixed.ipynb и гонишь сверху вниз.

2. Формат данных

2.1. Корпус документов

Поддерживаемые расширения:

```
DOC_EXTENSIONS = {".pdf", ".txt", ".md"}
```

То есть тебе нужно:

- положить в одну или несколько папок:
 - PDF,
 - TXT (UTF-8),
 - MD.
- указать пути к этим папкам/файлам в DATA_PATHS:

```
DATA_PATHS = [
    "/path/to/docs",
    "/path/to/other_docs/file1.pdf",
]
```

Как это дальше используется:

- Каждый файл → один или несколько Document с:
 - metadata["source"] — полный путь к файлу,
 - metadata["doc_id"] — имя файла без расширения,
 - плюс страница/прочие метаданные для PDF.
- Потом они режутся на чанки (CHUNK_SIZE, CHUNK_OVERLAP), к чанкам добавляются:
 - _chunk_id,
 - heading, section_path, type (table/text),
 - иерархия секций.

Если нужны другие форматы (.docx, .html), их нужно будет самому добавить в DOC_EXTENSIONS и в load_any() дописать обработку.

2.2. Формат

queries.json

Ноут умеет понимать несколько вариантов, но лучше придерживаться одного чёткого:

Рекомендуемый формат:

```
{
  "queries": [
    { "ID": 1, "question": "О чём раздел по продуктовой стратегии?" },
    { "ID": 2, "question": "Какие есть риски и ограничения?" }
  ]
}
```

- Корневой объект с полем "queries", внутри список.

- В каждом элементе:
 - ID — одно из: "ID", "id", "query_id".
 - текст — "question" или "query" или "text".

Ноут сам делает примерно так:

```
items = data.get("queries") or data.get("data") or data.get("items") or
data.get("questions")
```

Так что допустим и такой формат:

```
[
  { "id": 1, "text": "... " },
  { "id": 2, "text": "... " }
]
```

Главное, чтобы:

- ID был в одном из ожидаемых ключей,
- сам вопрос — тоже.

2.3. Формат

submission.csv

Функция `make_submission` собирает `DataFrame` и сохраняет `SUBMISSION_CSV`.

По умолчанию колонки такие:

- ID — число или строка, совпадает с тем, что было в `queries.json`;
- context — склеенный контекст для вопроса:

```
"[S1] ...\n\n[S2] ...\n\n...";
```

- answer — ответ LLM одной строкой;
- references — строка с JSON, например:

```
{"sections": ["Intro/Overview"], "pages": ["3", "4"]}
```

Если соревнование требует другие колоноки/названия — это правится в одной функции (см. ниже).

3. Как использовать ноут “по инструкции”

Шаг 0. Открыть и настроить CONFIG

В первой большой кодовой ячейке с комментарием CONFIG:

1. Пути к данным:

```
DATA_PATHS = [  
    "/path/to/your/corpus",  
]  
  
QUERIES_PATH = "/path/to/your/queries.json"  
SUBMISSION_CSV = "submission.csv"
```

- 1.
2. Настройки поиска:

```
PRE_K = 60  
FINAL_K = 10  
ADAPTIVE_TAU = 0.35  
MIN_K = 3  
MAX_K = 15  
USE_HIERARCHICAL_RETRIEVAL = True # можно выключить, если структура слабая
```

- 2.
3. Выбор модели (обычная / unsloth):

```
BASE_GEN_MODEL_NAME = "Qwen/Qwen2.5-1.5B-Instruct"  
UNSLOTH_GEN_MODEL_NAME = "unsloth/Qwen2.5-1.5B-Instruct"  
USE_UNSLOTH_MODEL = False # или True, если хочешь unsloth
```

3.
 - USE_UNSLOTH_MODEL = True → будет пытаться грузить 4-bit unsloth через BitsAndBytesConfig.
 - False → обычная HF-модель, float16 на GPU.
4. Остальные параметры (chunk size, overlap, модели эмбедингов/CE) можно оставить по умолчанию и подкрутить позже.

Шаг 1. Запустить всё до построения индексов

Дальше просто идёшь ячейка за ячейкой:

1. Импорты, фиксация сидов, device, логгер.
2. Загрузка корпуса: `corpus = load_corpus(DATA_PATHS)`.
3. Построение outline'ов, структура документов.
4. Чанкинг: `chunks = make_recursive_chunks(corpus, ...)`, присвоение `_chunk_id`, структура, тип блока.
5. Построение секций: `section_docs, section_to_chunk_ids = build_section_corpus_from_chunks(...)`.
6. Индексы:
 - `bm25 = build_bm25(chunks)`
 - `vs = build_faiss_index(chunks, ...)`
 - `bm25_sec, vs_sec` аналогично по секциям.
7. CrossEncoder:
 - либо:

```
ce = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2", device=device,
max_length=512)
```

7.
 -
 - либо дообучать через `train_cross_encoder_from_chunks`.
8. LLM — ячейка с `GEN_MODEL_NAME` → tok, mdl.

Шаг 2. Протестировать на одном вопросе

Например, в отдельной ячейке:

```
q = "КАКОЙ-ТО ТВОЙ ВОПРОС"

res = answer_one_question(
    question=q,
    chunks=chunks,
    bm25=bm25,
    vs=vs,
    ce=ce, # или ce=None, если не используешь
    adaptive_tau=ADAPTIVE_TAU,
    use_hierarchical=USE_HIERARCHICAL_RETRIEVAL,
)

print("ANSWER:\n", res["answer"])
print("\nCONTEXT:\n", res["context"][:1500])
print("\nREFERENCES:\n", res["references"])
```

Так ты проверишь:

- релевантен ли контекст,

- нет ли ошибок типа “нет индексов”, “модель не грузится”.

Шаг 3. Сделать submission

В самом конце ноута есть функция `make_submission`. Пример использования:

```
df_sub = make_submission(
    QUERIES_PATH,
    out_csv=SUBMISSION_CSV,
    adaptive_tau=ADAPTIVE_TAU,
)
df_sub.head()
```

После этого в рабочей директории появится файл `SUBMISSION_CSV` (например, `submission.csv`).

4. Как правильно менять сабмит под другую задачу

Вся логика формирования сабмита сосредоточена в одной функции: `make_submission`.

Внутри цикла по запросам там примерно:

```
rows.append({
    "ID": qid,
    "context": ctx_text,
    "answer": answer_text,
    "references": json.dumps(refs, ensure_ascii=False),
})
```

Если какая-то платформа требует:

- другие названия колонок — меняешь ключи в этом словаре:

```
rows.append({
    "QueryID": qid,
    "Response": answer_text,
    "Evidence": ctx_text,
})
```

- дополнительные поля (например, сырой JSON с документами, скор CE и т.п.) — просто добавляешь их:

```
rows.append({
    "ID": qid,
    "context": ctx_text,
    "answer": answer_text,
    "references": json.dumps(refs, ensure_ascii=False),
    "raw_ctx_docs": json.dumps([d.metadata for d in ctx_docs],
    ensure_ascii=False),
})
```

Главное правило: подгоняешь ровно этот словарь под формат, который от тебя ждут (Kaggle / внутренний валидатор / API).

5. Краткая шпаргалка

- Где запускать: Kaggle / Colab / локальный Jupyter — без разницы, главное правильно прописать пути и поставить зависимости.
- Формат корпуса: папки с .pdf/.txt/.md, пути в DATA_PATHS.
- Формат запросов: queries.json с массивом объектов и полями ID + question (или эквиваленты).
- Выбор модели:
 - USE_UNSLOTH_MODEL = False → обычный HF Qwen.
 - USE_UNSLOTH_MODEL = True → unsloth/Qwen2.5-1.5B-Instruct в 4-bit.
- Сабмит: формируется в make_submission; менять формат = править словарь rows.append({...}).

Сабмит

Но ⚠ они не запускаются автоматически — их нужно выполнить в нужном порядке (обычно в самом конце).

Что уже готово для сабмита

В ноуте есть все важные блоки:

Компонент	Статус	Где в ноуте
Загрузка и подготовка корпуса	✓	верхние ячейки
Выделение структуры документов	✓	блок про heading/outline
Чанкинг + метаданные	✓	make_recursive_chunks, _chunk_id

Компонент	Статус	Где в ноуте
BM25 + FAISS индексы	✓	build_bm25, build_faiss_index
Иерархический ретрив + CE	✓	hierarchical_retrieve, retrieve_with_rerank
Генерация ответа	✓	generate_answer, llm_generate
Формирование context	✓	build_context_for_submission
Формирование ссылок	✓	build_references_simple
Итоговый сабмит	✓	make_submission(...)

То есть логика сабмита полностью собрана и работает.

Что

обязательно

нужно выполнить вручную перед сабмитом

Порядок:

- 1 Заполняешь CONFIG
- 2 Прогоняешь весь RAG-пайплайн до загрузки модели
- 3 Загружаешь модель (LLM)
- 4 Тестируешь на 1 вопросе:

```
res = answer_one_question("пробный вопрос", chunks, bm25, vs, ce,
                           adaptive_tau=ADAPTIVE_TAU,
                           use_hierarchical=USE_HIERARCHICAL_RETRIEVAL)
print(res["answer"])
```

Если это успешно → идёшь к сабмиту:

- 5 Генерируешь submission.csv:

```
df_sub = make_submission(
    QUERIES_PATH,
    out_csv=SUBMISSION_CSV,
    adaptive_tau=ADAPTIVE_TAU,
```

```
)  
df_sub.head()
```

И всё 🎉

Файл SUBMISSION_CSV уже готов для отправки.

Проверка, что всё работает

После выполнения `make_submission`:

```
import os  
print("Файл существует?", os.path.exists(SUBMISSION_CSV))  
print("Размер (байт):", os.path.getsize(SUBMISSION_CSV))  
df_sub.head()
```

Если видишь:

- есть колонки `ID / context / answer / references`,
- ID совпадает с запросами,
- нет NaN

➡ Сабмит корректный.

На что обратить внимание перед запуском `make_submission`

Проверь	Почему важно
Пути: <code>DATA_PATHS</code> , <code>QUERIES_PATH</code> , <code>SUBMISSION_CSV</code>	иначе не найдёт данные или не запишет файл
Загрузилась ли LLM (mdl)	иначе ошибки при генерации
CE задан (ce) или стоит <code>ce=None</code>	обе опции нормальные, но должно быть явно
Хватает ли GPU-памяти	Qwen2.5-1.5B — всё же не крошка
У включённого <code>USE_UNSLOTH_MODEL</code> есть нужные зависимости	<code>bitsandbytes</code> + GPU требуются

Вывод

- ✓ Все функции для сабмита уже реализованы и готовы
 - ✓ Нужно только выполнить ноут по порядку и вызвать `make_submission`
 - ✓ Ты ничего больше не должен дописывать
-

Если хочешь, я могу:

- ◆ прямо здесь сформировать для тебя мини-чеклист перед нажатием “Run all”
- ◆ помочь проверить твои данные (`queries.json`, структура корпуса)
- ◆ протестировать один вопрос по твоему корпусу
- ◆ адаптировать формат сабмита под конкретный конкурс

Хочешь, сделаем контрольный прогон сабмита прямо сейчас на твоём ноуте? 🚀